

ENTERPRISE COMPUTING

Distributed Sys Design : CAP

LAB 2/W3

4TH YEAR - ENSIA

OBJECTIVES

- Observe a Single Point of Failure
- Experiment with Consistency vs Availability
- Understand CAP trade-offs through failure
- Start to build & Experiment Services using Java Spring Boot.

INDIVIDUAL EXERCISES

Task / Exercise C1 [20 Minutes + 5 minutes Discussion] (Introductory Service + SPOF) :

Given the following code :

```
from flask import Flask, request, jsonify
import requests
import uuid
from datetime import datetime

app = Flask(__name__)

PEER = "http://localhost:5001"
REQUIRE_REPLICATION = False
MODE = "AP" # "AP" or "CP"

documents = {} # In-memory document store
@app.route("/documents/add", methods=["GET"])
def add_document():
    title = request.args.get("title")
    content = request.args.get("content")
    if not title or not content:
        return jsonify({"error": "title and content required"}), 400
    doc_id = str(uuid.uuid4())
    document = {
        "id": doc_id,
        "title": title,
        "content": content,
        "created_at": datetime.utcnow().isoformat()
    }
    documents[doc_id] = document
    if REQUIRE_REPLICATION:
        try:
            requests.get(
                f"{PEER}/documents/replicate",
                params=document,
                timeout=1)
        except:
            return jsonify({"error": "Write rejected"}), 503
    return jsonify({
        "status": "document added",
        "document": document})

@app.route("/documents/replicate", methods=["GET"])
def replicate_document():
    doc_id = request.args.get("id")
    if not doc_id:
        return jsonify({"error": "invalid replication data"}), 400
    documents[doc_id] = {
        "id": doc_id,
        "title": request.args.get("title"),
        "content": request.args.get("content"),
        "created_at": request.args.get("created_at")}
    return jsonify({"status": "replicated"})
```

```
@app.route("/documents/list", methods=["GET"])
def list_documents():
    return jsonify(list(documents.values()))

if __name__ == "__main__":
    app.run(port=5000, debug=True)
```

1. Run just one instance/node of the code by executing the command : **python app.py**
2. Write and read data from the service provided. Students need to learn how to use Postman or Curl or other tools to create send requests.
3. **Imagine that this node is the only authority of data, and suddenly crashed (Terminate manually the service to simulate the crash)**
 - a. What would happen to other services relying on this node ? Facebook 2021 ?
 - b. Why the need to avoid SPOF
 - c. Write down the techniques of how to mitigate SPOF

Task / Exercise C2 [45 Minutes + 10 minutes discussion] (Service Availability vs. Consistency + CAP in Action) :

Part 1 : Redundancy Without Coordination.

1. Run two instances of the code on different ports :
 - a. Node A should run on port 5000
 - b. Node B runs on port 5001
2. Write to Node A whilst later list the documents from Node B.
3. The system is available. Why is it unsafe?

Part 2 : Availability vs. Consistency

1. Set the variable : `REQUIRE_REPLICATION = True`
2. Stop Node B (so it becomes unreachable) and write Node A
3. Discuss whether the consistency (CP) or availability (AP) is prioritized here when replicating the data or the code is not well structured ?

Part 3 : CAP in Action

Improve the code via using the MODE FLAG to work for the different two cases:

- a. DMS for a Banking App context where strict consistency must be maintained (CP)
- b. Chat/Messaging within a DMS where availability is a must.

Task / Exercise C3 [10 Minutes] (Exploring Enterprise Technologies) :

Students need to explore following technologies: identify whether it behaves mostly as CP or AP : etcd (Kubernetes datastore) Apache Cassandra, CouchDB and Apache Kafka

ASSIGNMENT (A1 : Due for evaluation next lab)

Students need to prepare for the next lab where they should migrate the Python code provided to them in Ex01 to Java Spring Boot and run it on their machines locally.

What's required:

1. Migrate the Code given to you to Java Spring Boot
2. Run only a single instance created using Spring Boot locally.
3. Test it using Postman, Curl or other tools but not a browser.

OPTIONAL EXERCISES

Task / Exercise P1

Without implementation, give recommendations for exercise 1 to include the following features:

- Possibility to replicate data to a large number of additional nodes in the future.
- Integrate a load balancer to avoid SPOF

Task / Exercise P2

How to integrate the strategy design pattern to improve the structure of Ex2 - Part 3